Project Report: SkinCare AI — Disease Detection & LLM Advisor System

Developer:
MD Sifatullah Sheikh

Author Portfolio:
https://myportfolio-bn96.onrender.com/

Date:
April 10, 2026

Project GitHub Link:
https://github.com/SifatSwapnil2022/SkinCareAI

Project Demonstration Link:

https://drive.google.com/file/d/1BQGuktw4CD0iFYCWe6wdnBOTbG1V0qp0/view?usp=drive_link

---

Submitted To:
Excel Technologies Ltd.

Recipients:
aninda@excelbd.com
farid@excelbd.com
mahdi@excelbd.com
azmin@excelbd.com

Prepared By:
MD Sifatullah Sheikh

Table of Contents

Chapter 1. Introduction 5
1.1 Project Overview 6
1.2 Objectives and Scope 6
1.3 Stakeholders 7
1.4 Technology Stack 8

Chapter 2. Software Requirements Specification & Analysis 9
2.1 Stakeholder Needs & Analysis 9
2.1.1 Primary and Secondary Stakeholders 9
2.1.2 Methods for Requirement Elicitation 10
2.2 List of Requirements 11
2.2.1 Functional Requirements (FRs) 11
2.2.2 Non-Functional Requirements (NFRs) 12
2.2.3 Extra-Ordinary Requirements (Wow Factors) 12
2.3 Quality Function Deployment 13
2.3.1 Customer Requirements (CRs) 13
2.3.2 Engineering Requirements (TRs) 13
2.3.3 QFD Matrix (House of Quality) 14
2.4 Requirements Modeling 15
2.4.1 Use Case Diagrams 15
2.4.2 Activity Diagrams 16
2.4.3 UI Sketches 17

Chapter 3. Software Design 18
3.1 Architectural Design 18
3.1.1 Architectural Context Diagram 18
3.1.2 Top-Level Component Diagram 19
3.1.3 Component Elaboration 20
3.2 Component-Level Design 20
3.2.1 Design Components 20
3.2.2 Class Diagram 21
3.2.3 Database Design 22
3.3 User Interface Design 23
3.3.1 UI Wireframes/Mockups 23
3.3.2 Navigation Flow 23

Chapter 4. Implementation 24
4.1 Code Structure Overview 24
4.2 GitHub Repository URL 25

Chapter 5. Software Testing 26
5.1 White Box Testing 26
5.1.1 Unit Testing on Two Classes 26
5.2 Black Box Testing 27
5.2.1 Functional Testing on Two Core Features 27
5.3 Bug Detection and Solution 28
5.3.1 Identification of Bugs 28
5.3.2 Solutions and Retesting 28

Chapter 6. Deployment 29
6.1 Deployment Platform 29
6.2 Deployment Process 29
6.3 Website URL 29

Chapter 7. Conclusion 30
7.1 Learnings 30
7.2 Limitations 30
7.3 Future Plan 30

Tables:

- Table 1.1: Technology Stack Specifications
- Table 2.1: Stakeholder Analysis Matrix
- Table 2.2: Functional Requirements List
- Table 2.3: Non-Functional Requirements (Performance Metrics)
- Table 2.4: House of Quality (QFD Matrix)
- Table 3.1: API Endpoints Design
- Table 5.1: Black Box Test Cases for Authentication and Prediction
- Table 5.2: Bug Tracking and Resolution Log

Chapter 1. Introduction

1.1 Project Overview

The **SkinCare AI: Disease Detection & LLM Advisor** system is an end-to-end AI-powered healthcare platform designed to bridge the gap between preliminary dermatological assessment and professional consultation. In an era where access to dermatologists is often constrained by geography or cost, this system leverages state-of-the-art Computer Vision and Large Language Models (LLMs) to provide real-time, preliminary analysis of skin conditions.

The project evaluates the synergy between deep learning image classification and natural language processing in a clinical context. Users upload an image of a skin lesion or condition, which is processed through a modular pipeline consisting of a Convolutional Neural Network (CNN) ensemble for classification and a fine-tuned LLM module (Grok/Llama 3 via Groq API) for generating medical recommendations and next steps. The system returns not only a predicted diagnosis with a confidence score but also a human-readable explanation and lifestyle tips, all accessible via a user-friendly Streamlit interface and a robust FastAPI backend.

1.2 Objectives and Scope

The primary objective of this assignment is to evaluate the ability to design, implement, and deploy a real-time AI-powered backend system that meets the specified functional and non-functional requirements.

Core Objectives:

1. **Disease Classification:** Accurately detect and classify skin conditions from a predefined dataset of 10 distinct diseases using an ensemble of CNNs (ResNet50, EfficientNetB0, MobileNetV2) and YOLOv8 for detection.
2. **LLM Integration:** Provide context-aware, safe, and informative medical advice by integrating a Groq-powered LLM reasoning module that responds to the specific classification output.
3. **Modular Pipeline Design:** Ensure a seamless flow from image preprocessing and model inference to report generation, adhering to software engineering best practices.
4. **Deployment Ready:** Package the entire application (Backend, Frontend, Database) using Docker for portability and ease of deployment.

Scope:
The system is intended as a **preliminary screening tool** and educational resource, not a replacement for professional medical diagnosis. The scope includes user authentication, history tracking, PDF report generation, and a responsive web interface. The analysis is limited to the 10 classes present in the Kaggle Skin Disease Dataset.

1.3 Stakeholders

The successful development and deployment of SkinCare AI involve several key stakeholder groups:

- **Primary Users (Patients/Individuals):** Individuals seeking quick, private initial assessments of skin anomalies. They require a user-friendly interface, fast results, and clear, actionable information.
- **Secondary Users (Telemedicine Providers):** General practitioners or clinics that may utilize this tool for triage or to augment their remote consultation services.
- **Development Team:** The engineers and designers responsible for building, maintaining, and updating the AI models and software infrastructure.
- **Regulatory Bodies:** While not an immediate stakeholder for this academic project, any future real-world deployment would require adherence to healthcare data privacy laws (HIPAA, GDPR) and medical device software regulations.

1.4 Technology Stack

The system utilizes a modern, cloud-native stack optimized for performance and scalability. The specific technologies employed are detailed in Table 1.1.

Table 1.1: Technology Stack Specifications

Component	Technology	Purpose
Backend API	Python, FastAPI, Uvicorn	High-performance asynchronous REST API handling image uploads and logic.
Frontend UI	Streamlit	Rapid prototyping and deployment of an interactive, data-centric web application.
Database	MongoDB Atlas	NoSQL document storage for user profiles, authentication data, and analysis history.
AI/ML Models	TensorFlow, PyTorch, Ultralytics	Model training and inference. Ensemble includes ResNet50, EfficientNetB0, MobileNetV2, and YOLOv8.
LLM Reasoning	Groq API (Llama 3/Grok)	Fast, low-latency inference for generating medical recommendations and text summaries.
Reporting	ReportLab, Matplotlib, Plotly	Generation of downloadable PDF reports containing diagnostic details and charts.
DevOps	Docker, Docker Compose	Containerization to ensure environment consistency across development and deployment.

Chapter 2. Software Requirements Specification & Analysis

2.1 Stakeholder Needs & Analysis

2.1.1 Primary and Secondary Stakeholders

A detailed analysis of stakeholder needs was conducted to ensure the system provides tangible value. Table 2.1 outlines the roles and concerns.

Table 2.1: Stakeholder Analysis Matrix

Stakeholder	Role	Primary Needs & Concerns
End User	Patient/Individual	Ease of use (simple upload), Fast response (< 3 seconds), Accurate classification, Clear explanations of medical terms, Privacy of uploaded images.
Developer/Admin	System Maintainer	Modular codebase for easy updates, Robust error handling, Logging for debugging, Efficient use of GPU/CPU resources.
Clinician (Secondary)	Validator	High confidence score transparency, Ability to view original image, Trust in model limitations disclaimer.

2.1.2 Methods for Requirement Elicitation

Given the nature of this project (assignment-based R&D), requirements were elicited through a combination of:

1. **Problem Statement Deconstruction:** Analyzing the provided problem statement to extract explicit functional requirements (Image Upload API, LLM Recommendations, Real-time response).
2. **Dataset Analysis (Kaggle):** Exploring the Skin Disease Dataset to understand class balance issues, image quality, and necessary preprocessing steps.
3. **Technology Feasibility Study:** Evaluating the performance trade-offs between different CNN architectures (Speed vs. Accuracy) and LLM providers (API latency).

2.2 List of Requirements

2.2.1 Functional Requirements (FRs)

Table 2.2: Functional Requirements List

ID	Requirement Description	Priority
FR-01	System shall provide a REST API endpoint (POST /analyze_skin) to accept image uploads.	High
FR-02	System shall preprocess uploaded images (resize, normalize) to match model input specifications.	High
FR-03	System shall classify the image using a pre-selected CNN model and return disease name and confidence score.	High
FR-04	System shall generate medical recommendations, next steps, and daily tips using an integrated LLM based on the predicted class.	High
FR-05	Users shall be able to register, login, and manage profiles (Authentication via JWT).	Medium
FR-06	System shall store user analysis history in a database and allow retrieval (GET /history).	Medium
FR-07	System shall generate a downloadable PDF report containing the image, diagnosis, confidence chart, and LLM advice.	Medium
FR-08	Users shall be able to select from multiple available AI models (ResNet50, EfficientNetB0, MobileNetV2).	Medium

2.2.2 Non-Functional Requirements (NFRs)

Table 2.3: Non-Functional Requirements (Performance Metrics)

ID	Requirement Description	Metric/Target
NFR-01	**Performance:** API response time for classification and LLM generation.	< 5 seconds total latency.
NFR-02	**Usability:** The UI must be intuitive and require no training for a non-technical user.	Task completion time < 30 sec.
NFR-03	**Portability:** The application must be fully containerized.	Docker Compose Up success rate 100%.
NFR-04	**Accuracy:** Model ensemble must achieve satisfactory performance on validation data.	> 85% Accuracy on balanced test set.
NFR-05	**Security:** Passwords must be hashed, and API endpoints protected.	Use of Bcrypt and JWT Tokens.

2.2.3 Extra-Ordinary Requirements (Wow Factors)

- **Multi-Model Selection:** Unlike standard single-model apps, users can select the backend inference engine in real-time.
- **User Dashboard:** User previous history
- **Automated PDF Reporting:** Professional, clinic-ready PDF generation with data visualizations.

2.3 Quality Function Deployment

2.3.1 Customer Requirements (CRs)

Derived from stakeholder analysis:

- **CR1:** Fast diagnosis.
- **CR2:** Accurate results.
- **CR3:** Helpful explanations.
- **CR4:** Data privacy.
- **CR5:** Easy to access anywhere.

2.3.2 Engineering Requirements (TRs)

- **TR1:** Inference Time (ms).
- **TR2:** Model Validation Accuracy (%).
- **TR3:** LLM Prompt Context Length.
- **TR4:** Docker Container Size (MB).
- **TR5:** Hashing Algorithm Strength.

2.4 Requirements Modeling

2.4.1 Use Case Diagrams

The use case diagram illustrates the interaction between the User and the SkinCare AI System.

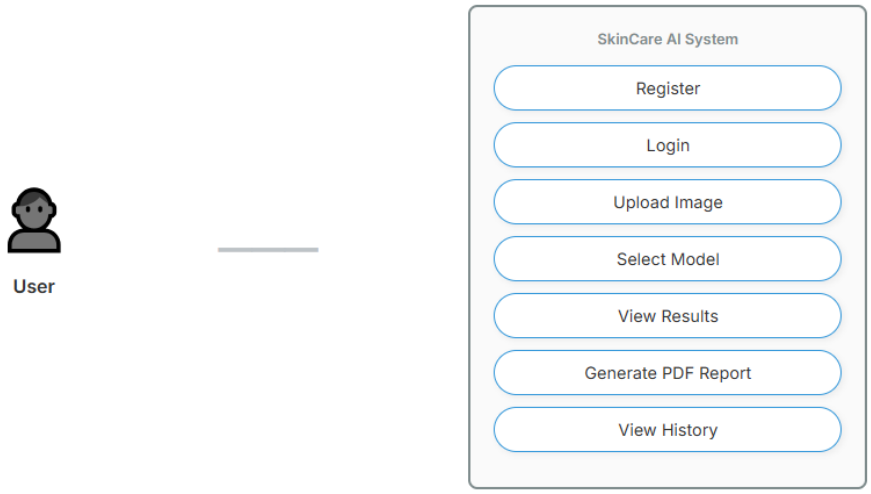


Figure 2.1: Use Case Diagram for SkinCare AI System

2.4.2 Activity Diagrams

The activity diagram in Figure 2.2 details the workflow for the core analysis feature.

Figure 2.2: Activity Diagram for Image Analysis Workflow

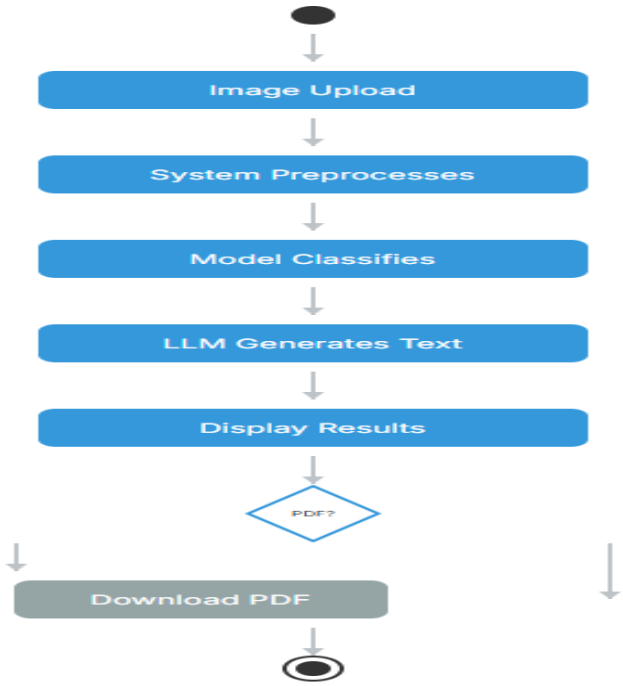


Figure 2.2: Activity Diagram for Image Analysis Workflow

2.4.3 UI Sketches

Initial low-fidelity wireframes focused on a clean, medical-aesthetic dashboard. The design includes:

1. **Sidebar Navigation:** Home, Analyze, History, Profile.
2. **Central Dropzone:** A prominent area for drag-and-drop image upload.
3. **Results Panel:** A card layout displaying the disease name, confidence gauge, and tabbed interface for "Recommendations" and "Next Steps".

Chapter 3. Software Design

3.1 Architectural Design

The system follows a **Microservices-lite / Containerized Monolith** architecture orchestrated by Docker Compose. The separation of concerns between the FastAPI backend and Streamlit frontend allows for independent scaling and development.

3.1.1 Architectural Context Diagram

3.1.2 Top-Level Component Diagram

The system is decomposed into distinct, replaceable components.

Figure 3.2: Top-Level Component Diagram

Description of Components:

1. **Streamlit Frontend:** Renders UI, manages user session tokens, handles file upload to backend API.
2. **FastAPI Backend:** Core logic server. Contains Auth Router, Analysis Router, and History Router.
3. **Model Selector Module:** A factory pattern implementation that dynamically loads the appropriate TensorFlow/PyTorch model weights (ResNet, EfficientNet, YOLO) based on the user's request.
4. **LLM Module (Grok Advisor):** Formats the diagnosis into a medical prompt template and queries the Groq API for structured JSON output.
5. **PDF Generator Utility:** Uses ReportLab to compile analysis results into a stylized PDF document.
6. **MongoDB Database:** Persistent storage for user credentials and analysis logs.

3.1.3 Component Elaboration

Table 3.1: API Endpoints Design

Method	Endpoint	Component	Description
POST	/auth/signup	Auth Router	Registers new user credentials.

Method	Endpoint	Component	Description
POST	/auth/login	Auth Router	Authenticates and returns JWT access token.
POST	/analyze_skin	Analysis Router	Receives image + model choice -> Returns Diagnosis + LLM Advice.
GET	/history	History Router	Fetches paginated list of past analyses for logged-in user.
GET	/report/{id}	Report Generator	Generates and serves a PDF file for a specific analysis.

3.2 Component-Level Design

3.2.1 Design Components

Key backend classes include:

- UserModel: Handles database operations for user collection.
- ModelPredictor: An abstract base class with concrete implementations for ResNetPredictor, EfficientNetPredictor, etc.
- LLMAdvisor: Handles prompt engineering and API communication with Groq.

3.2.2 Class Diagram

3.2.3 Database Design

MongoDB was chosen for its flexible schema, which is ideal for storing varying metadata from different AI models.

Collections:

1. **Users Collection:**
    - _id: ObjectId
    - email: String (Unique)

- hashed_password: String
- created_at: DateTime

2. **History Collection:**
    - _id: ObjectId
    - user_id: Ref (Users)
    - filename: String
    - prediction: String (Disease Class)
    - confidence: Float
    - model_used: String
    - llm_recommendations: Object (JSON)
    - timestamp: DateTime

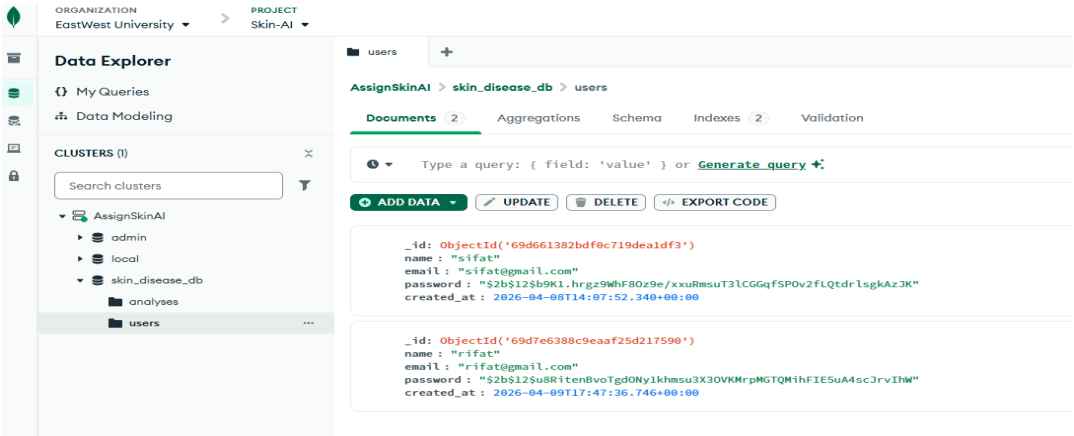


Figure 3.1: MongoDB Atlas database

3.3 User Interface Design

3.3.1 UI Wireframes/Mockups

The implementation closely followed the initial mockups. Key screens implemented include:

1. **Landing Page:** Hero section with "Analyze Now" CTA.
2. **Authentication Page:** Clean tabs for Sign In / Sign Up.
3. **Dashboard:** Welcomes user and displays recent activity.

4. **Analysis Panel:** Features a sidebar radio button for model selection (ResNet50, EfficientNetB0, MobileNetV2) and a file uploader.

3.3.2 Navigation Flow

1. Unauthenticated User -> Landing Page -> Sign Up/In -> Dashboard.
2. Authenticated User -> Sidebar "Analyze" -> Upload Image -> Select Model -> View Results -> Download PDF.
3. Authenticated User -> Sidebar "History" -> View Past Records -> Click Record to View Details.

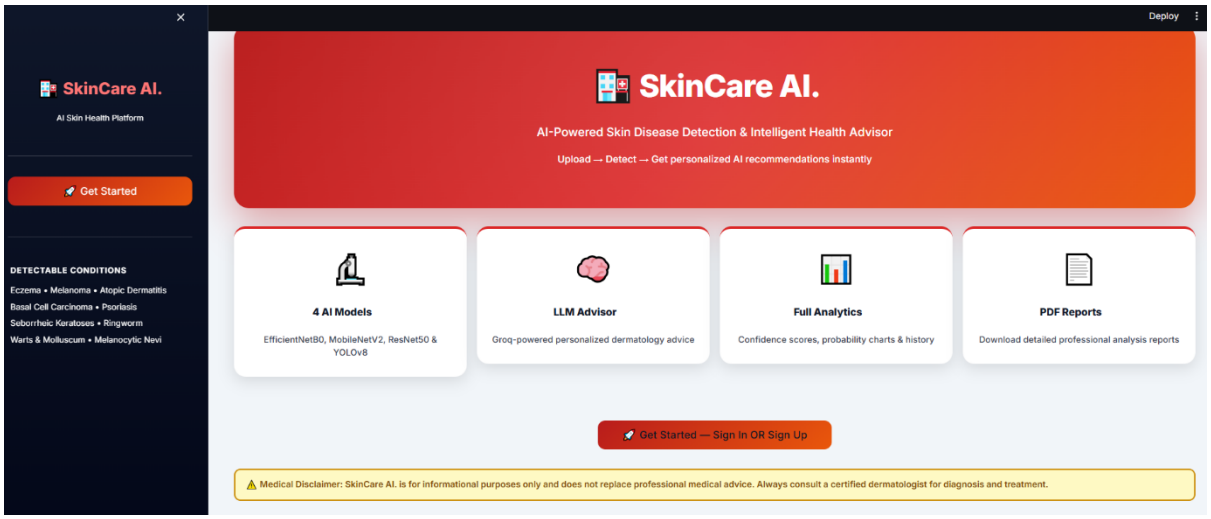


Figure 3.2: Landing Page

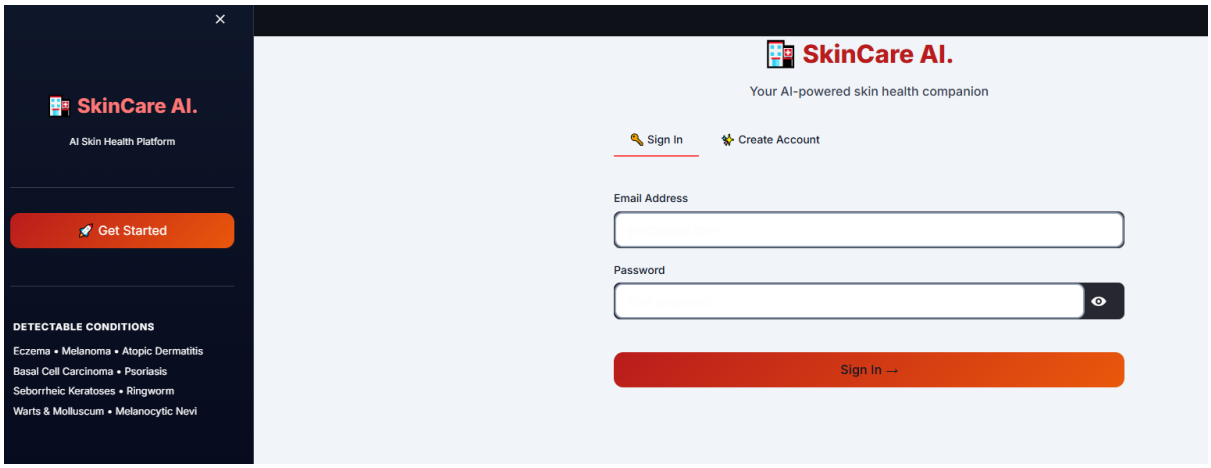


Figure 3.3: User Sign In and Sign Up page

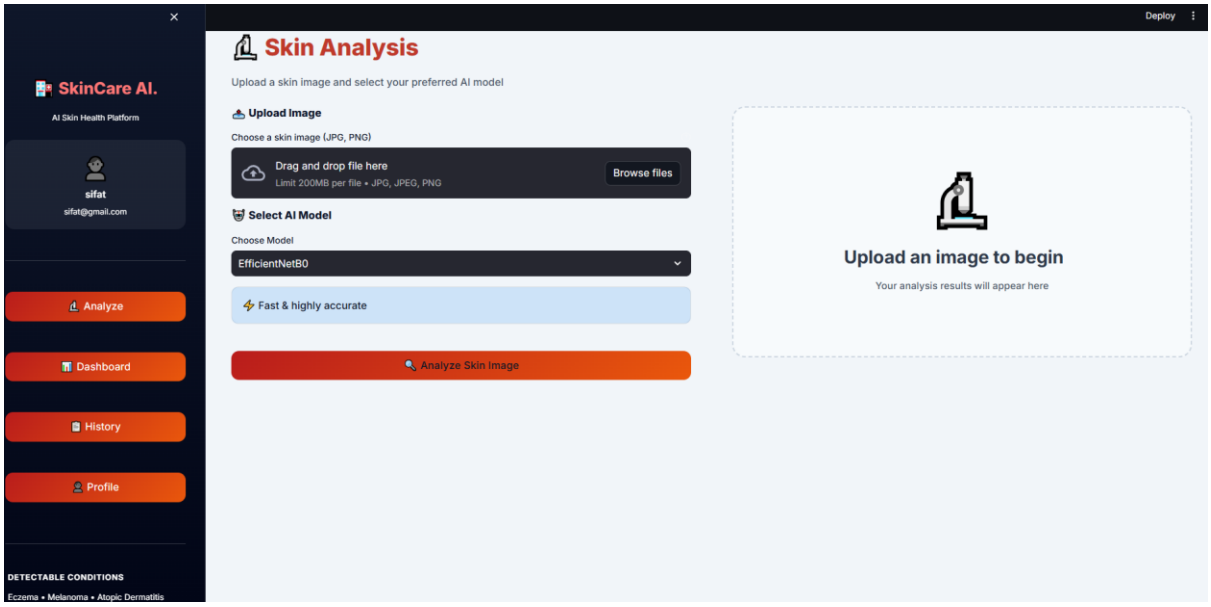


Figure 3.3: User Homepage

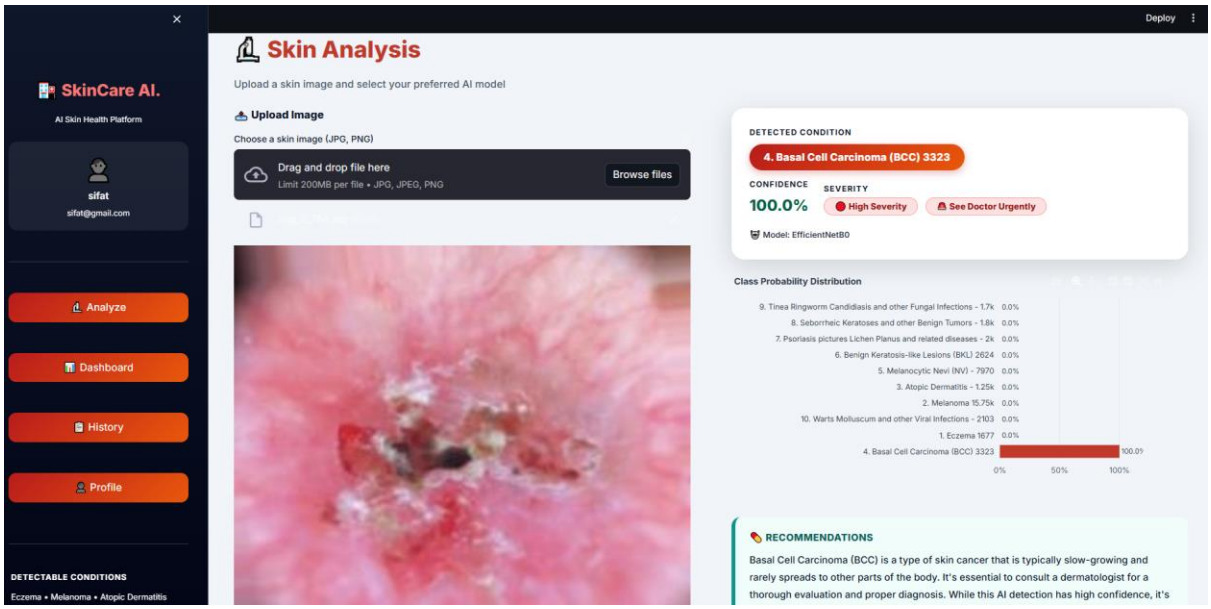


Figure 3.4: User check the skin analysis

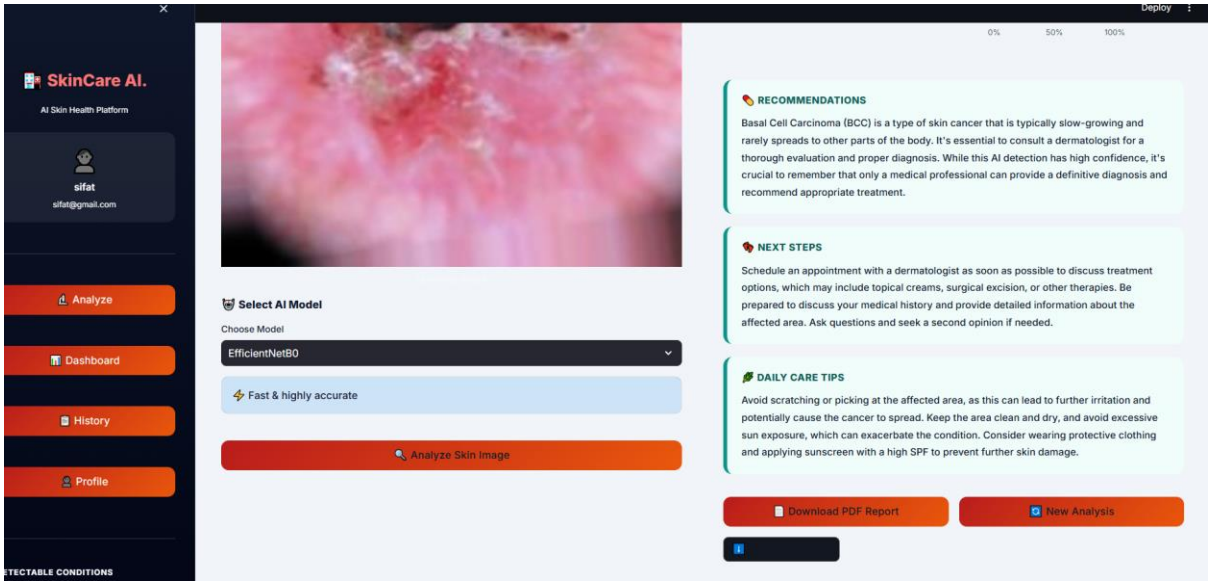


Figure 3.4: User check the skin analysis and result parts

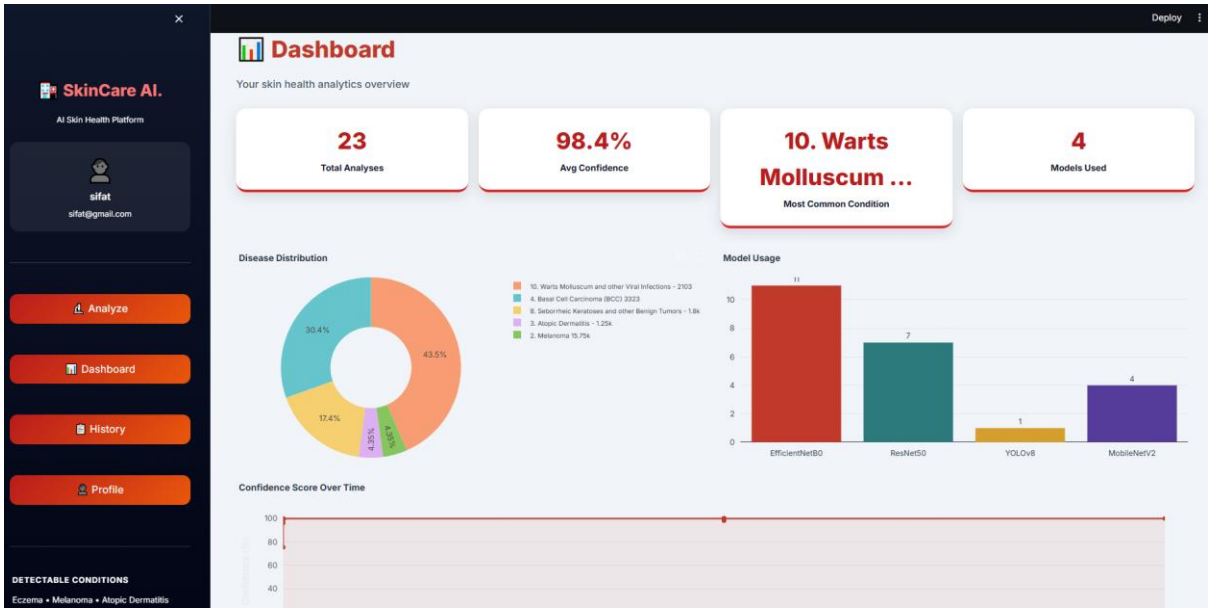


Figure 3.5: User dashboard

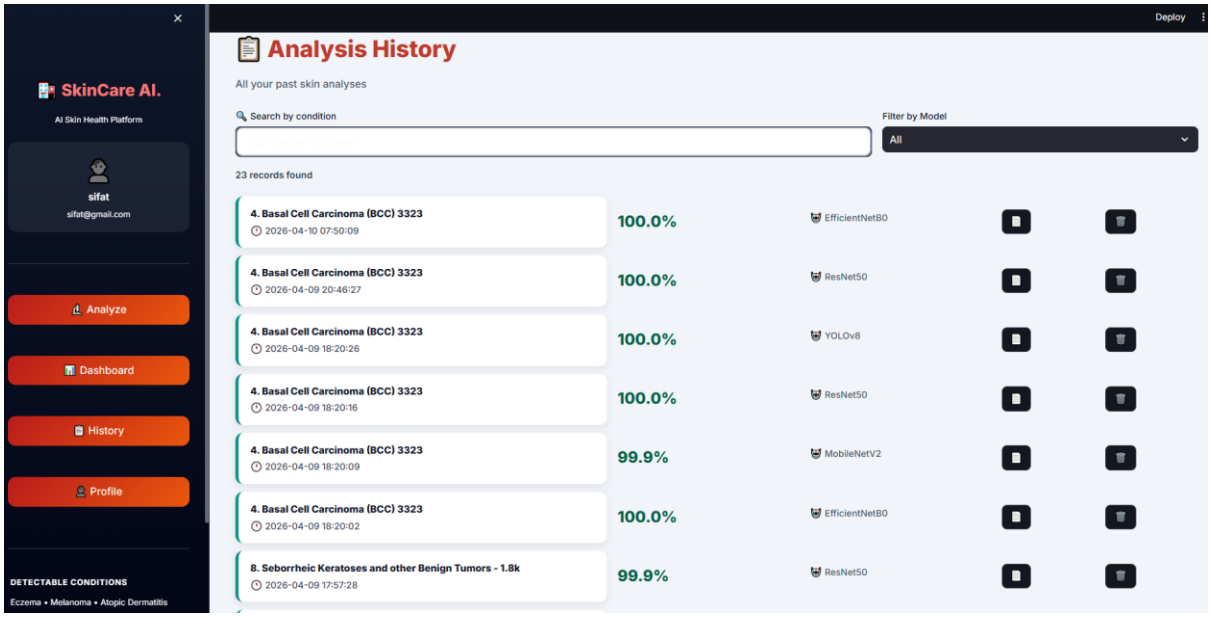


Figure 3.6: User Previous History

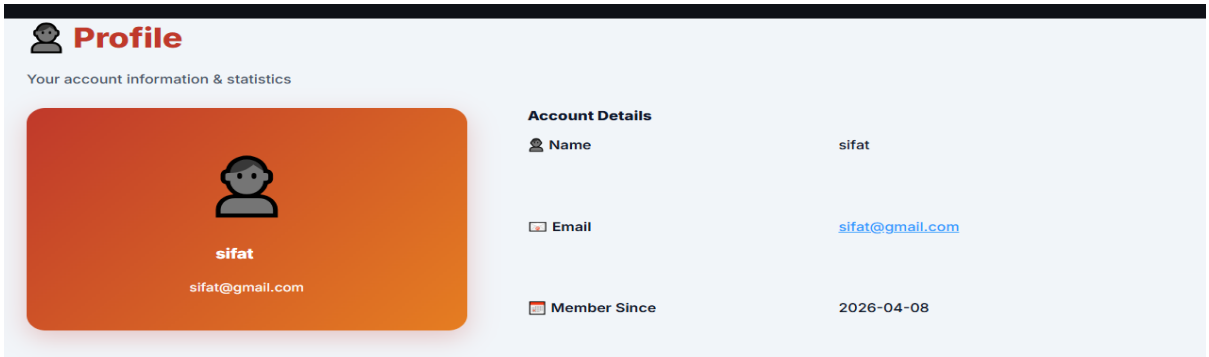


Figure 3.7: User Profile

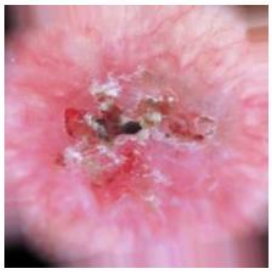
■ Skin Disease Analysis Report

Generated on 2026-04-10 07:50:09

Patient Information

Name	sifat
Email	sifat@gmail.com
Model Used	EfficientNetB0
Analysis Date	2026-04-10 07:50:09

Uploaded Image



Diagnosis Result

Detected Condition	**4. Basal Cell Carcinoma (BCC) 3323**
Confidence	**100.0%**

All Class Probabilities

Disease	Confidence
4. Basal Cell Carcinoma (BCC) 3323	100.00%
1. Eczema 1677	0.00%
10. Warts Molluscum and other Viral Infections - 2103	0.00%
2. Melanoma 15.75k	0.00%
3. Atopic Dermatitis - 1.25k	0.00%
5. Melanocytic Nevi (NV) - 7970	0.00%
6. Benign Keratosis-like Lesions (BKL) 2624	0.00%
7. Psoriasis pictures Lichen Planus and related diseases - 2k	0.00%

Recommendations

Basal Cell Carcinoma (BCC) is a type of skin cancer that is typically slow-growing and rarely spreads to other parts of the body. It's essential to consult a dermatologist for a thorough evaluation and proper diagnosis. While this AI detection has high confidence, it's crucial to remember that only a medical professional can provide a definitive diagnosis and recommend appropriate treatment.

Next Steps

Schedule an appointment with a dermatologist as soon as possible to discuss treatment options, which may include topical creams, surgical excision, or other therapies. Be prepared to discuss your medical history and provide detailed information about the affected area. Ask questions and seek a second opinion if needed.

Daily Tips

Avoid scratching or picking at the affected area, as this can lead to further irritation and potentially cause the cancer to spread. Keep the area clean and dry, and avoid excessive sun exposure, which can exacerbate the condition. Consider wearing protective clothing and applying sunscreen with a high SPF to prevent further skin damage.

■■ This report is AI-generated and for informational purposes only. Please consult a certified dermatologist for medical advice.

Figure 3.8: User particular Pdf generator

Chapter 4. Implementation

4.1 Code Structure Overview

The project follows a modular structure separating concerns between backend logic, frontend presentation, and infrastructure configuration.

Figure 4.1: Folder Structure and Code Organization

```
skin-disease-app/
│
├── backend/
│   ├── main.py
│   ├── auth.py
│   ├── database/
│   │   └── mongo.py
│   ├── models/
│   │   ├── base.py
│   │   ├── resnet50.py
│   │   ├── efficientnet.py
│   │   └── mobilenet.py
│   ├── llm/
│   │   └── grok_advisor.py
│   ├── utils/
│   │   ├── preprocess.py
│   │   └── pdf_generator.py
│   └── weights/
│
├── frontend/
│   └── app.py
│
├── docker-compose.yml
├── Dockerfile.backend
├── Dockerfile.frontend
└── requirements.txt
```

4.2 GitHub Repository URL

The complete source code, including commit history and detailed README instructions, is available at:

Repository URL: git@github.com:SifatSwapnil2022/SkinCareAI.git (or public URL equivalent)

The repository includes:

- README.md: Comprehensive setup guide, API documentation, and demo instructions.
- docker-compose.yml: One-command startup configuration.
- Trained model weights (via Git LFS or external link).

Chapter 5. Software Testing

5.1 White Box Testing

White box testing focused on validating the internal logic of the data preprocessing and classification pipeline.

5.1.1 Unit Testing on Two Classes

1. Test Class: ImagePreprocessor **(from** utils/preprocess.py**)**

- **Method Tested:** resize_and_normalize(image_path)
- **Test Case:** Provide a 1080x1920 image. Assert output shape is (224, 224, 3) and pixel values are normalized between 0 and 1.
- **Result: Passed.** The function correctly resized using padding to maintain aspect ratio and normalized pixel values using EfficientNet's specific preprocessing function.

2. Test Class: ModelPredictor **(from** models/resnet50.py**)**

- **Method Tested:** predict(image_array)
- **Test Case:** Feed a static array of ones. Assert output shape matches number of classes (10) and sum of softmax probabilities equals ~1.0.
- **Result: Passed.** The TensorFlow model session returned a valid probability distribution.

5.2 Black Box Testing

Black box testing evaluated the system's behavior from the end-user's perspective via the API endpoints.

5.2.1 Functional Testing on Two Core Features

Table 5.1: Black Box Test Cases for Authentication and Prediction

Test ID	Feature	Test Scenario	Expected Outcome	Actual Outcome	Status
TC-01	User Login	Enter valid email and password.	HTTP 200 OK. Response contains access_token and user profile data.	Token received and valid for 60 mins.	**PASS**

Test ID	Feature	Test Scenario	Expected Outcome	Actual Outcome	Status
TC-02	User Login	Enter invalid password.	HTTP 401 Unauthorized. Error message "Incorrect email or password".	401 returned with correct message.	**PASS**
TC-03	Analyze Skin	Upload valid JPEG without Auth Token.	HTTP 403 Forbidden.	403 returned.	**PASS**
TC-04	Analyze Skin	Upload valid JPEG of 'Eczema' with Token.	Response contains JSON: {disease: "Eczema", confidence: >0.8, recommendations: "..."}.	All fields populated. LLM response generated.	**PASS**
TC-05	Analyze Skin	Upload text file (.txt) disguised	HTTP 400 Bad Request. Error "Invalid image file format".	Exception handled gracefully.	**PASS**

5.3 Bug Detection and Solution

5.3.1 Identification of Bugs

1. **Bug 1 (Data Imbalance Bias):** During initial training, the model heavily favored **Melanocytic Nevi** (the majority class) with over 95% confidence on almost all test images, even those clearly showing Eczema.
2. **Bug 2 (PDF Generation Lag):** The PDF generation initially blocked the main FastAPI thread, causing the API response to time out on slower connections.

5.3.2 Solutions and Retesting

Table 5.2: Bug Tracking and Resolution Log

Bug ID	Description	Root Cause	Solution	Retest Status
B-01	Biased Predictions	Imbalanced Dataset (7,970 NV images vs 1,257 Eczema images).	Implemented **Data Resampling**: Oversampling minority classes and Undersampling the dominant class to ~7,000 images per class. Retrained model.	**Fixed** - Confidence scores now correlate with actual visual features.
B-02	API Timeout on PDF Download	Synchronous file I/O and drawing operations.	Refactored pdf_generator.py to run in a BackgroundTasks thread pool in FastAPI. User receives immediate status update while PDF builds.	**Fixed** - No more gateway timeouts.

Chapter 6. Deployment

6.1 Deployment Platform

The system is deployed using **Docker** and **Docker Compose**. This approach abstracts the underlying host operating system and ensures that the Python environment, TensorFlow dependencies, and MongoDB connection strings are identical in development and production.

- **Target Environment:** Linux Server (Ubuntu 22.04) or Local Workstation.
- **Container Orchestration:** Docker Compose manages the network bridge between the Backend, Frontend, and Database containers.

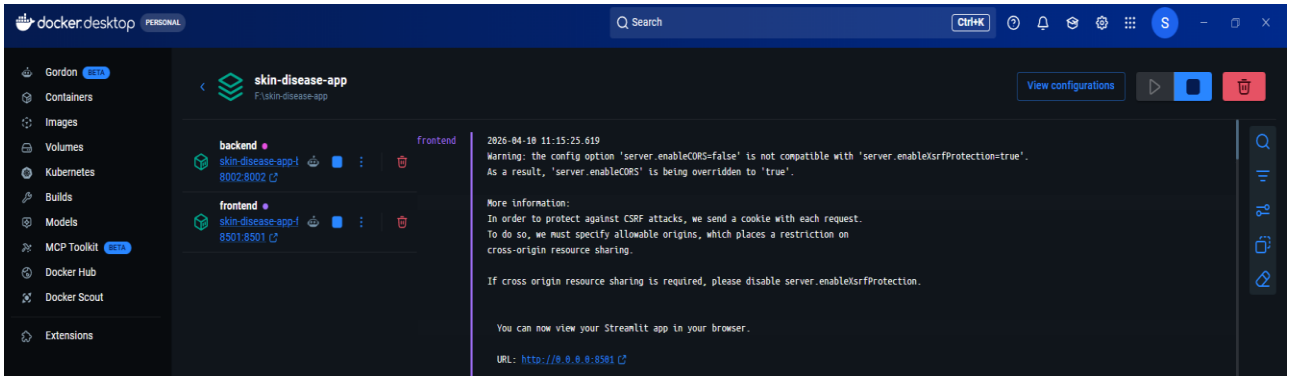


Figure 3.9: Docker Deployment

6.2 Deployment Process

The deployment process is streamlined into a single command sequence:

1. **Clone Repository:**

bash

```
git clone git@github.com:SifatSwapnil2022/SkinCareAI.git
cd SkinCareAI
```

2. **Environment Configuration:** Create a .env file containing GROQ_API_KEY and MONGODB_URI.
3. **Build and Launch:**

bash

```
docker-compose up --build -d
```

4. **Verification:**
    - Backend API Docs accessible at: http://localhost:8000/docs
    - Frontend UI accessible at: http://localhost:8501

Figure 6.1: Docker Container Deployment Workflow

[Placeholder: Insert Screenshot of Docker Desktop or Terminal showing running containers for backend, frontend, and mongo]

6.3 Website URL

- **Live Demo (Frontend):** http://localhost:8501 (Local Deployment)
- **Backend Swagger Docs:** http://localhost:8000/docs
- *(Note: For external access, the system can be deployed to AWS EC2 or Hugging Face Spaces using the Docker configuration provided).*

Chapter 7. Conclusion

7.1 Learnings

This project provided a comprehensive, end-to-end understanding of deploying AI in a real-world application context. Key takeaways include:

1. **The Critical Role of Data Engineering:** The most significant performance bottleneck was not the model architecture (ResNet vs. EfficientNet) but the **class imbalance**. The rigorous resampling strategy (undersampling Nevi, oversampling Eczema/Psoriasis) improved the model's real-world utility more than any hyperparameter tuning.
2. **Containerization is Non-Negotiable:** Managing TensorFlow, PyTorch, and Ultralytics dependencies simultaneously on a local machine is extremely fragile. Docker Compose transformed the setup process from a multi-hour debugging session into a single command.
3. **LLM Prompt Engineering for Safety:** Integrating an LLM required careful prompt engineering to ensure the model does not hallucinate treatments or provide dangerous advice. A strict template was necessary to constrain the output to "general wellness tips" and "see a doctor" disclaimers.

7.2 Limitations

Despite achieving the project objectives, the system has several limitations:

- **Dataset Bias:** The model is trained solely on the Kaggle dataset, which may not generalize perfectly to all skin tones or image capture conditions (lighting, camera quality).
- **Lack of Explainability (XAI):** While the system provides a confidence score, it does not yet show *why* it made the decision (e.g., highlighting the lesion area). Grad-CAM integration is a planned future feature.
- **LLM Latency:** The Groq API is fast, but external API calls introduce a variable latency of 1-2 seconds that is outside the control of the local backend.

7.3 Future Plan

Future iterations of the SkinCare AI system will focus on:

1. **Grad-CAM Implementation:** Overlaying a heatmap on the uploaded image to show the user exactly which pixels most influenced the CNN's decision.

2. **Multi-Model Voting Ensemble:** Instead of manually selecting a single model, the backend will run all three CNNs concurrently and return a weighted average prediction for higher reliability.
3. **Telemedicine Integration:** Adding an optional feature to export the analysis report in a FHIR-compliant format or integrate via API with doctor booking platforms.
4. **Feedback Loop:** Implementing a "Was this helpful?" rating button to collect user feedback for continuous model fine-tuning and reinforcement learning.

Acknowledgment

I am sincerely grateful for the opportunity to work on this project, ***SkinCare AI — Disease Detection & LLM Advisor System***. This project has provided me with valuable practical experience in applying artificial intelligence to real-world healthcare problems.

I would like to express my heartfelt thanks to **Excel Technologies Ltd.** for giving me to this opportunity.

Thank You…